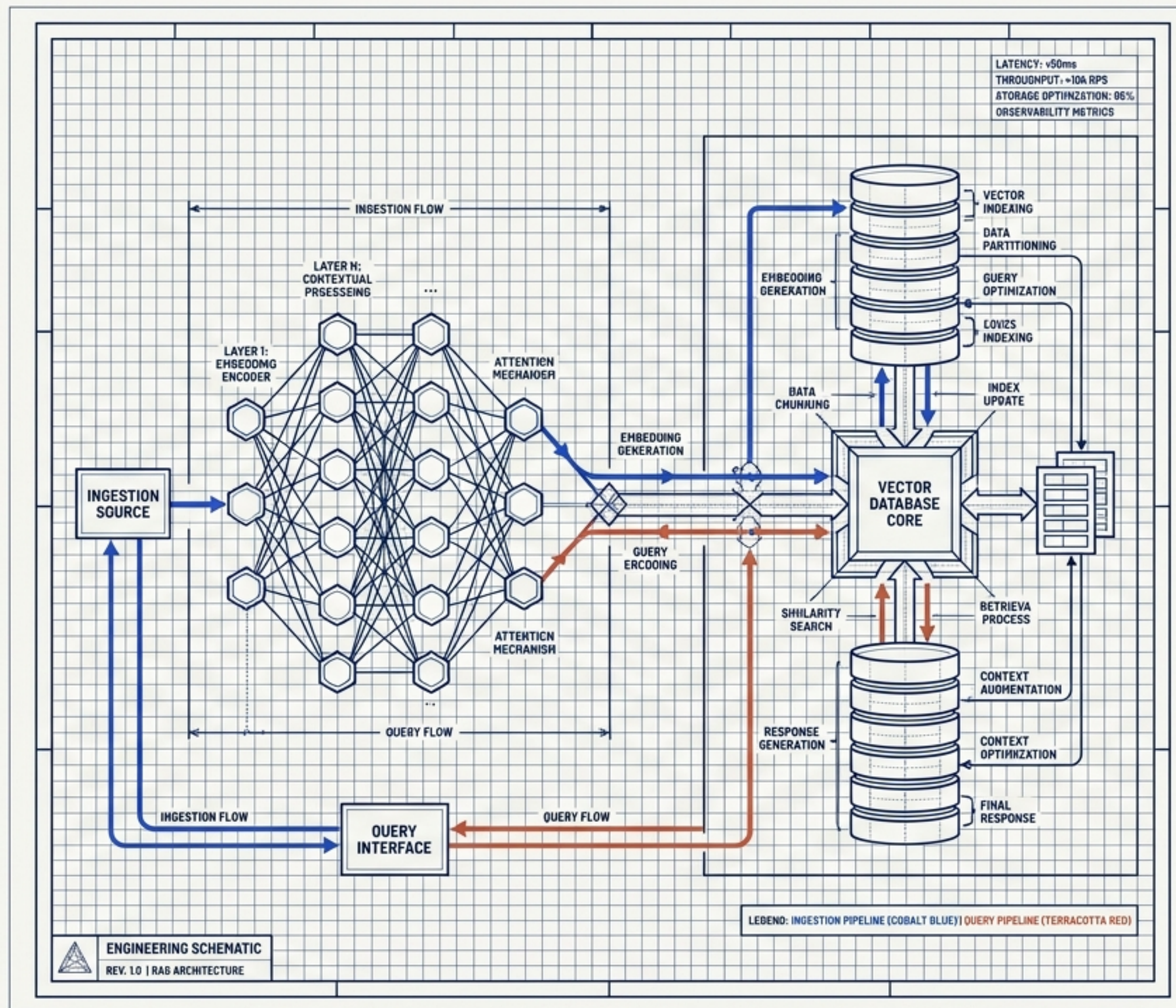


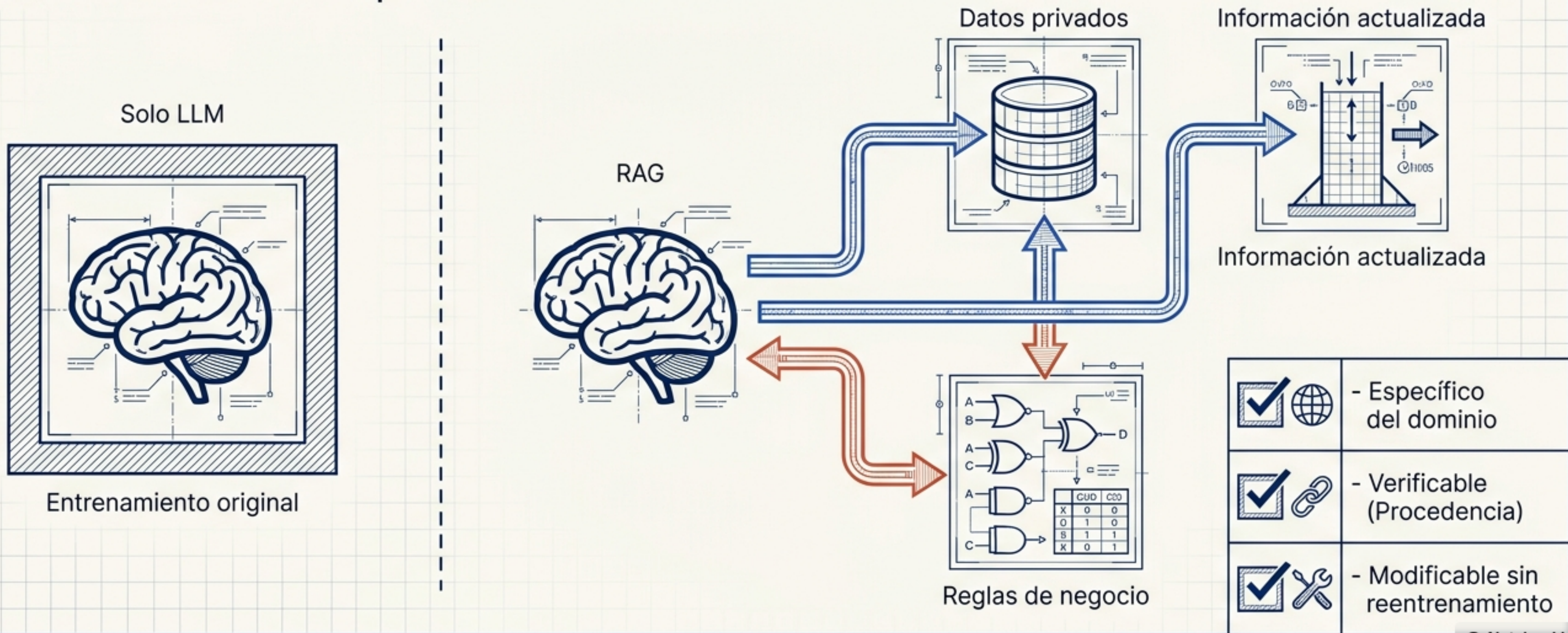
RAG en Producción

De un prototipo local a una arquitectura escalable, segura y observable. Una guía definitiva para ingenieros.



El modelo no necesita recordarlo todo

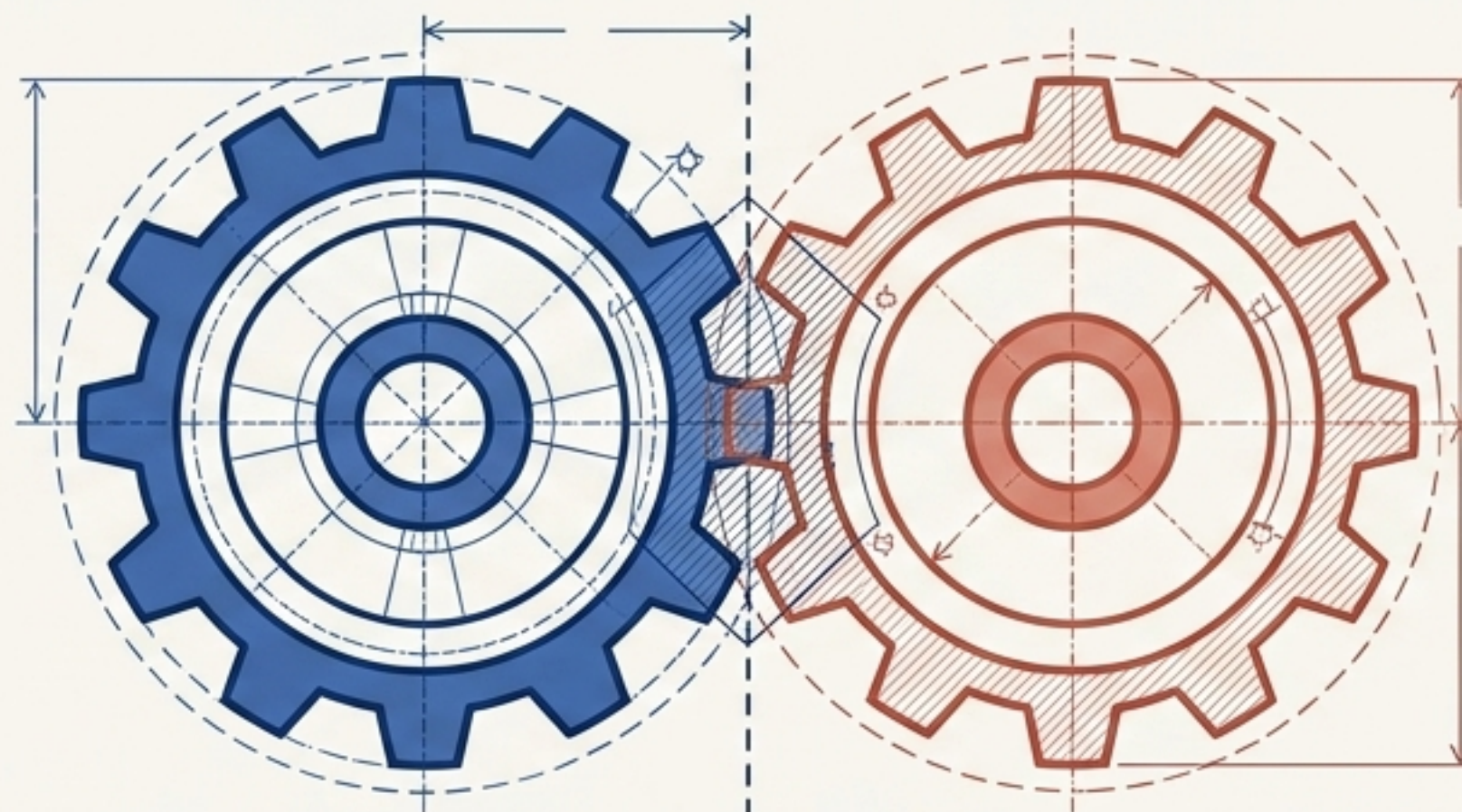
RAG (Retrieval-Augmented Generation) cambia el comportamiento del modelo. No reemplaza al LLM, lo complementa permitiéndole consultar una fuente externa antes de responder.



El núcleo del sistema son dos pipelines fuertemente acoplados

Pipeline de Ingestión

Prepara la base de conocimiento



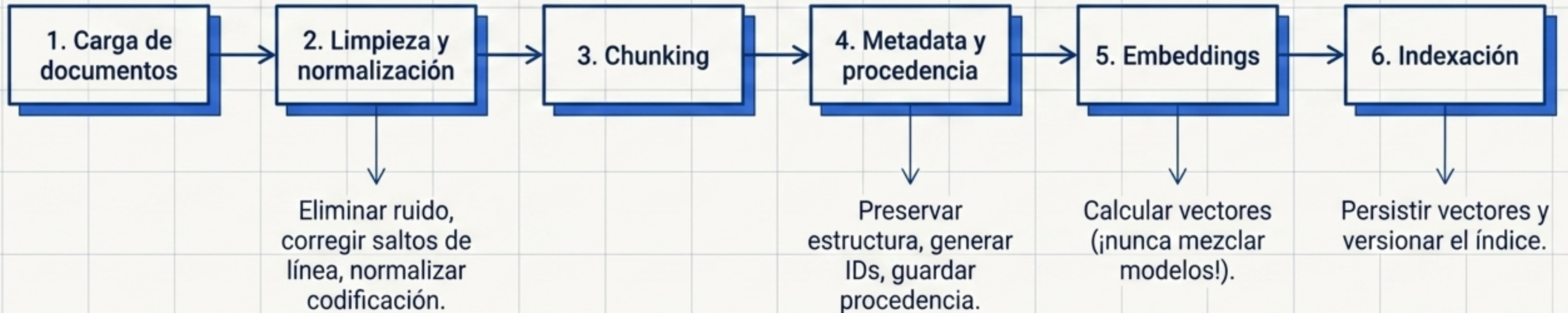
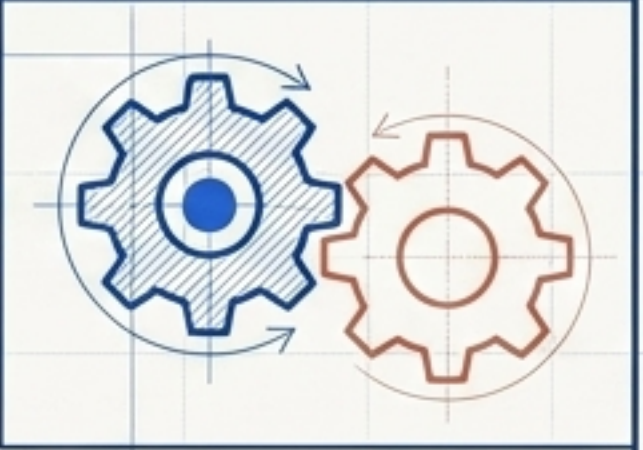
Pipeline de Consulta

Responde preguntas de usuarios

Cause & Effect Dependency Web

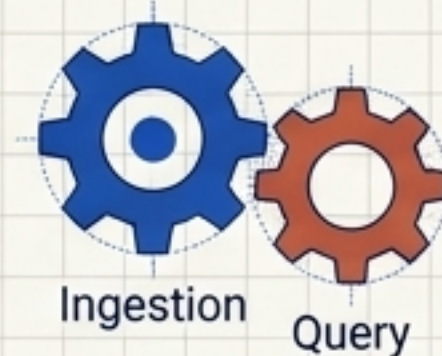


Anatomía del pipeline de datos

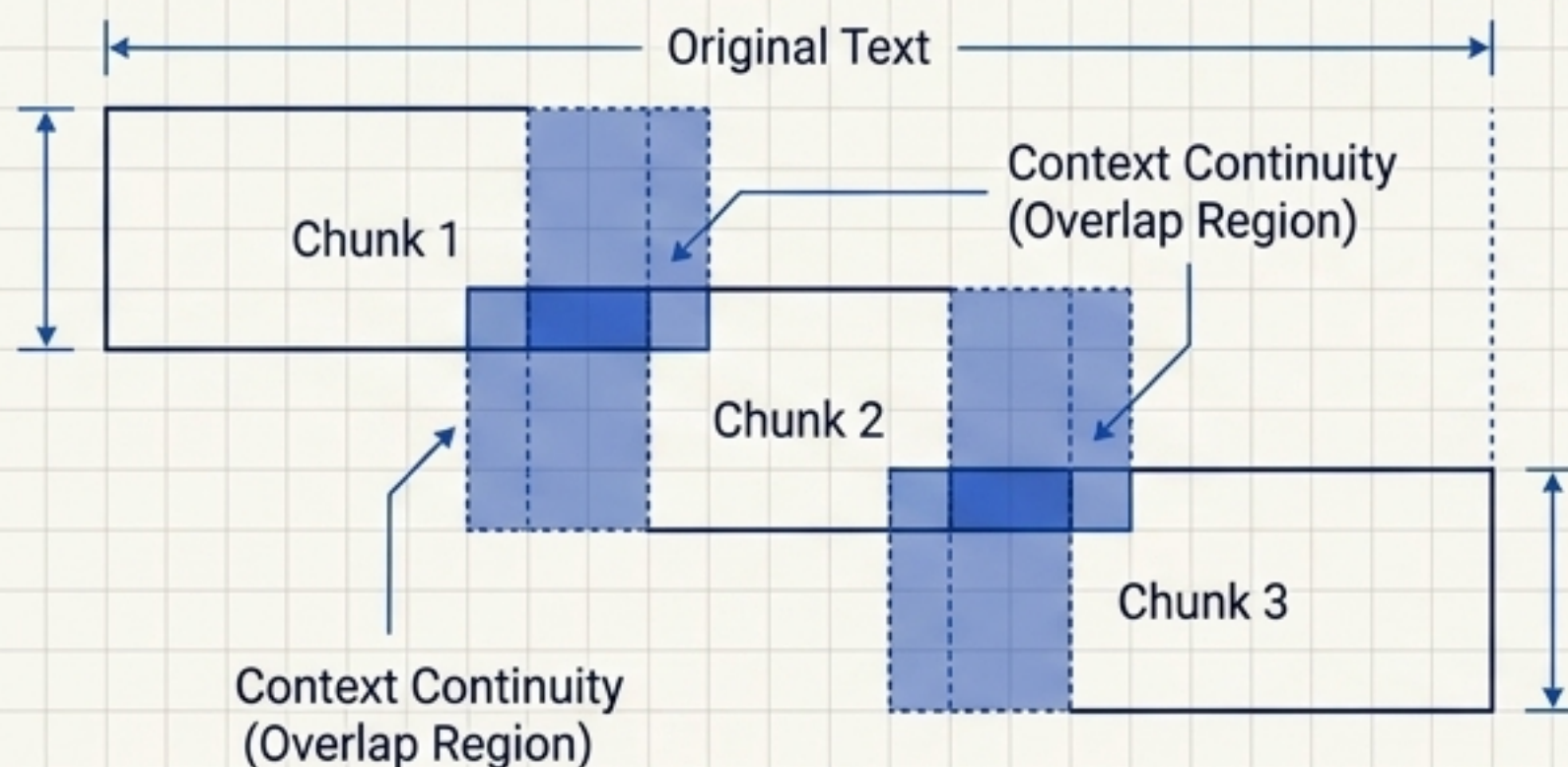


Regla de ejecución: Este pipeline se ejecuta al cargar documentos, al cambiar una fuente, o al modificar la estrategia de chunking/embeddings.

La calidad de la consulta nace en el chunking



Chunking Mechanics



Trade-off

Chunk grande = más contexto pero menos precisión.
Chunk pequeño = alta precisión pero riesgo de perder el sentido global.

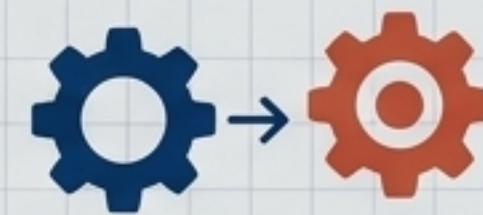
Metadata Architecture

```
{
  "source": "https://company.com/docs/manual_v2.pdf",
  "chunk_index": 42,
  "document_version": "2.1.0-alpha",
  "permissions": [
    "role_engineer",
    "role_admin"
  ],
  "content_vector": "[...]",
  "metadata_hash": "a1b2c3d4e5f6g7h8"
}
```

- source: URL o ruta del archivo.
- chunk_index: Posición original en el texto.
- document_version: Para invalidación de caché.
- permissions: Roles de acceso (RBAC/ABAC).

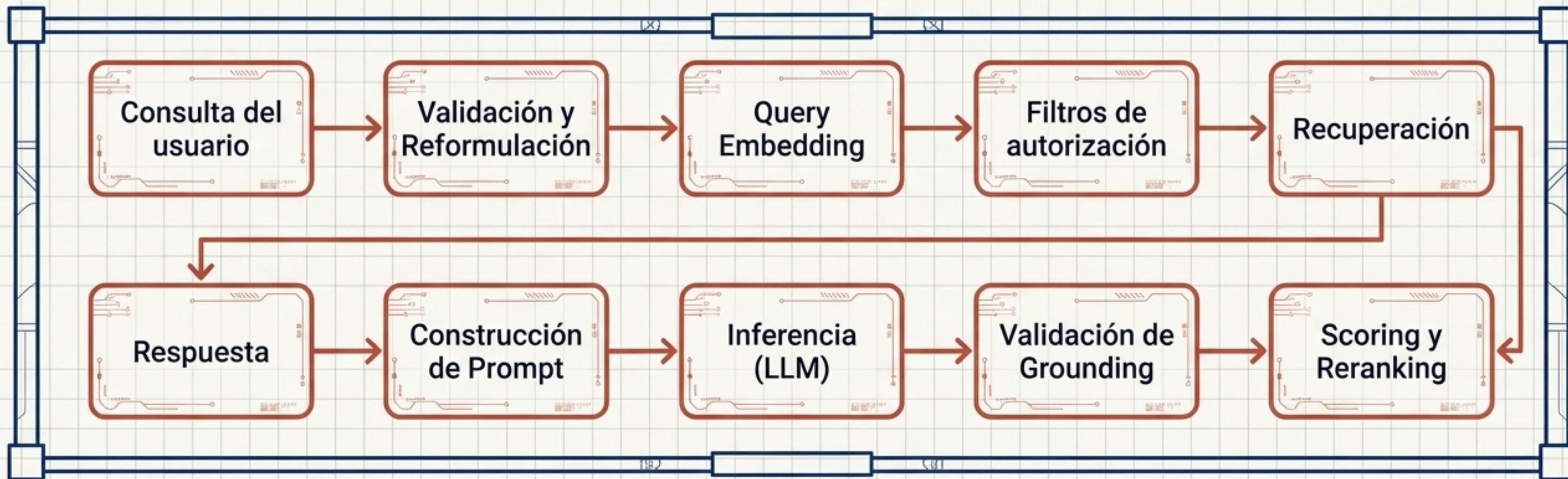
El ciclo de vida de una consulta

Minimap



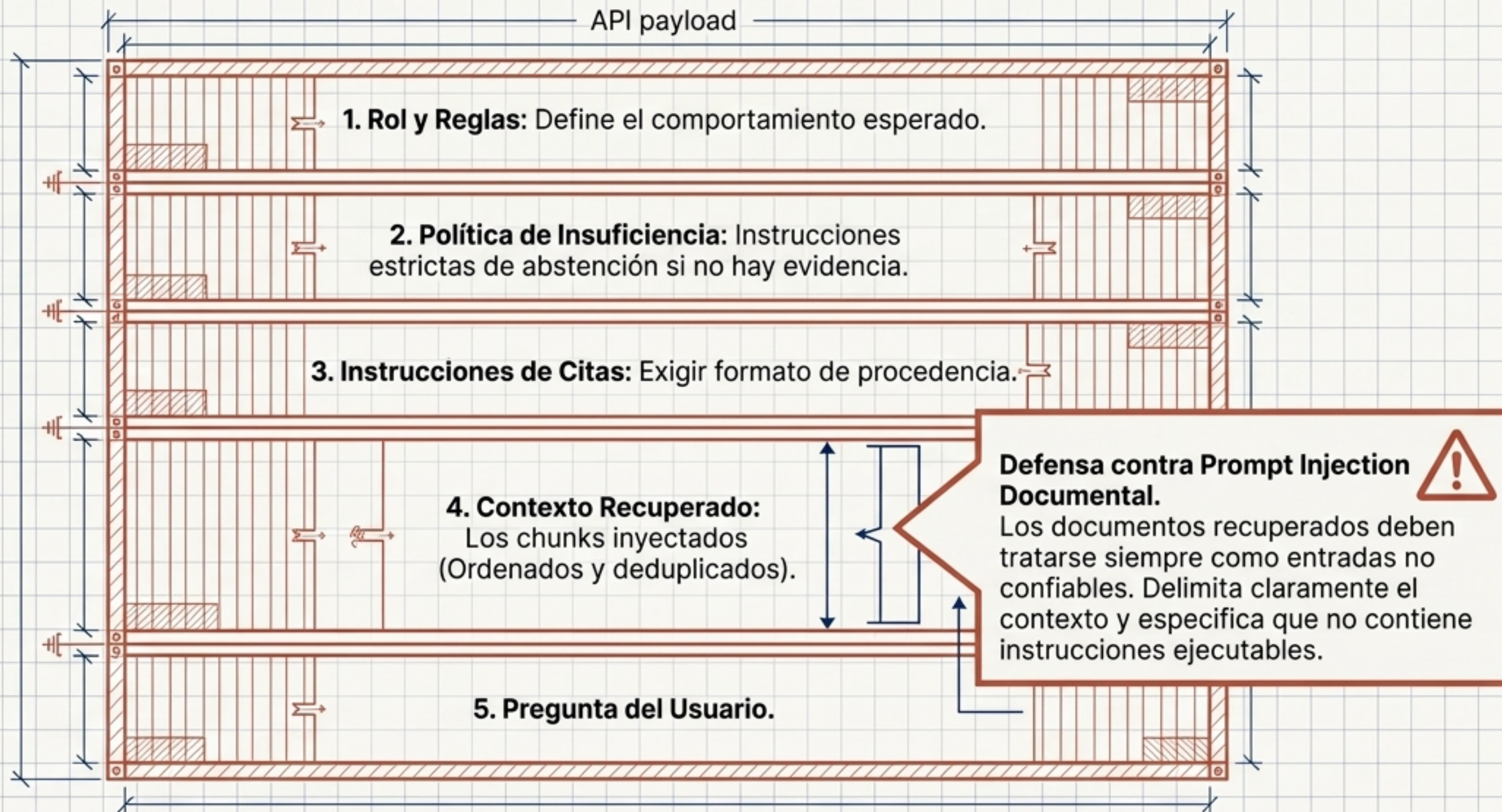
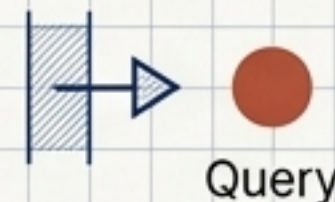
Ingestion

Query



La recuperación es la barrera principal. Si el sistema no recupera la información correcta aquí, ninguna cantidad de procesamiento en el LLM podrá generar una respuesta confiable.

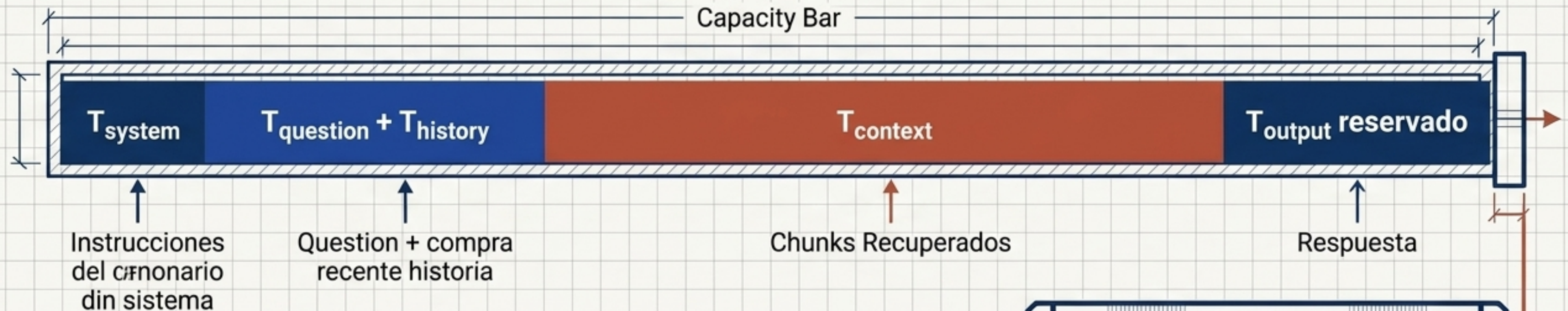
Construcción estructural del Prompt RAG



Token Budgeting: El contexto es un recurso limitado



$$T_{total} \leq T_{window}$$



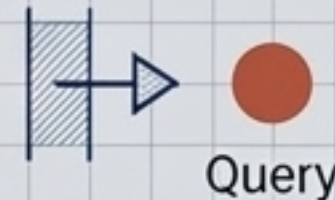
Prioridad del Presupuesto (Conserva primero en este orden):

1. Instrucciones críticas
2. Pregunta
3. Fuentes más relevantes (Chunks por orden)
4. Metadata necesaria
5. Historial reciente



Arquitectura en dos etapas: Búsqueda y Reranking

Minimap



Candidatos (Embeddings)

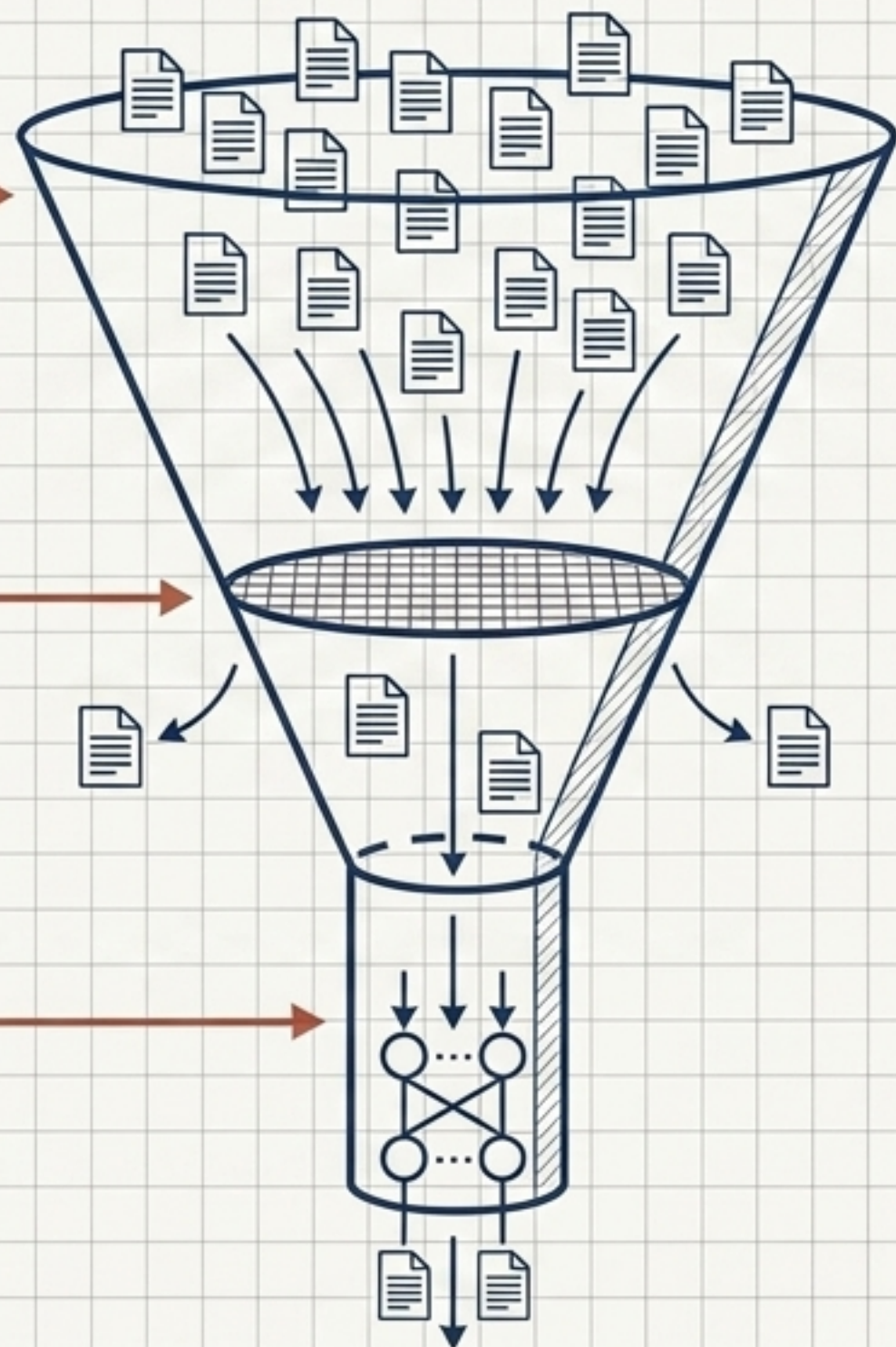
Búsqueda vectorial rápida e independiente.

Threshold

Elimina resultados de baja señal.
(¡Calibrar con datos reales!)

Top-K (Reranker Cross-Encoder)

Evaluación conjunta, acoplada y altamente precisa.

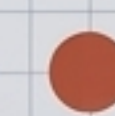


$$Score_{final} = \alpha S_{semantic} + \\ + \beta S_{reranker} + \\ + \gamma S_{business}$$

Reglas de negocio ($S_{business}$): El ranking productivo incluye prioridad, prioridad oficial, fecha, idioma y permisos.

Mitigación de Alucinaciones y Grounding

Minimap



Query

Anatomía de las Alucinaciones

Retrieval Failure

El sistema no recupera la información correcta.



Generation Failure

Contexto correcto, modelo respondió mal.



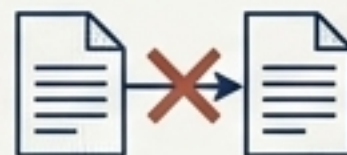
Source Failure

La fuente es incorrecta o está desactualizada.



Attribution Failure

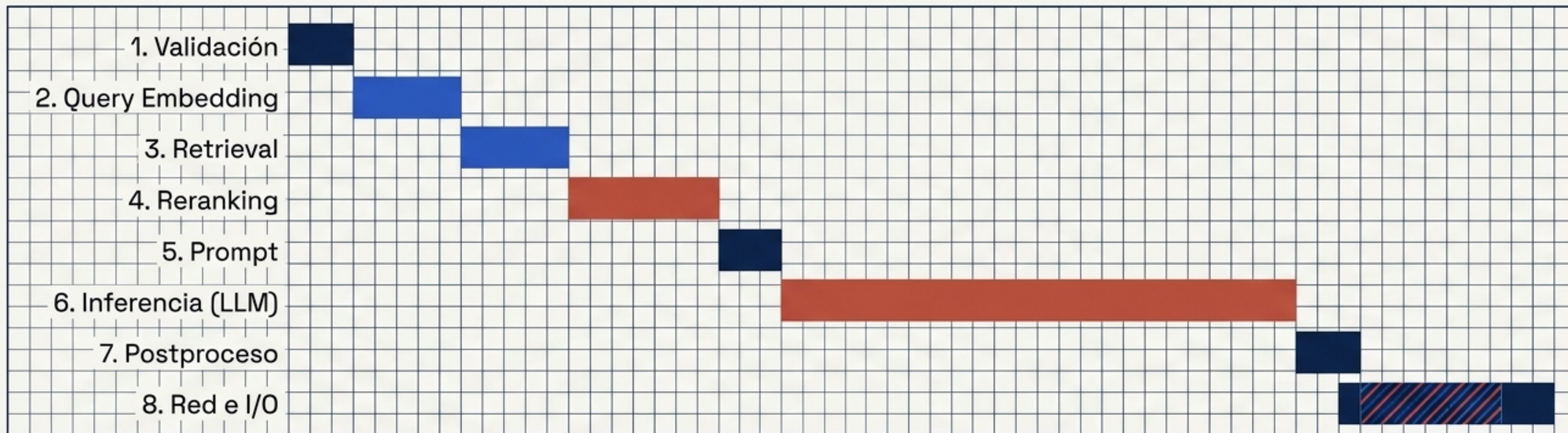
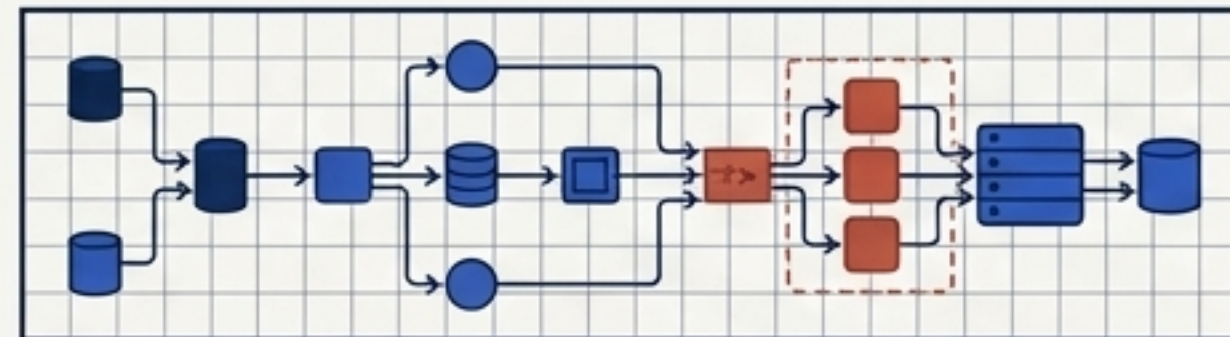
Respuesta correcta, pero cita fuente equivocada.



Para garantizar que la respuesta esté "grounded" (respaldada por contexto):

- Implementar políticas de abstención estrictas.
- Requerir respuestas **estructuradas** para **validación posterior**.
- Extraer evidencia y detectar contradicciones **antes de enviar al usuario**.

Latencia End-to-End: Optimizando los Percentiles



$$L_{total} = L_{embedding} + L_{retrieval} + L_{reranking} + L_{prompt} + L_{LLM} + L_{post} + L_{network}$$

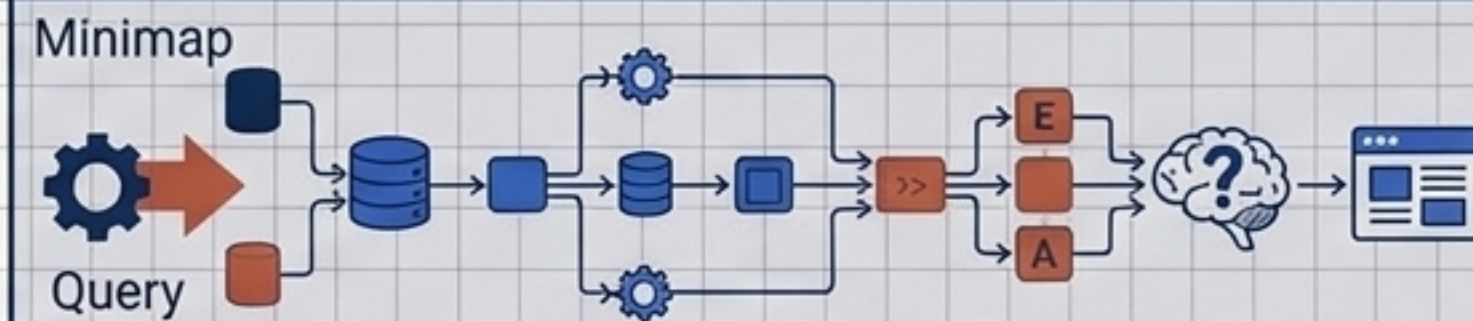


Nunca optimices solamente el promedio (p50). Diseña para el p95 (la mayoría de solicitudes incluyendo la cola lenta) y monitorea el p99 (casos extremos).

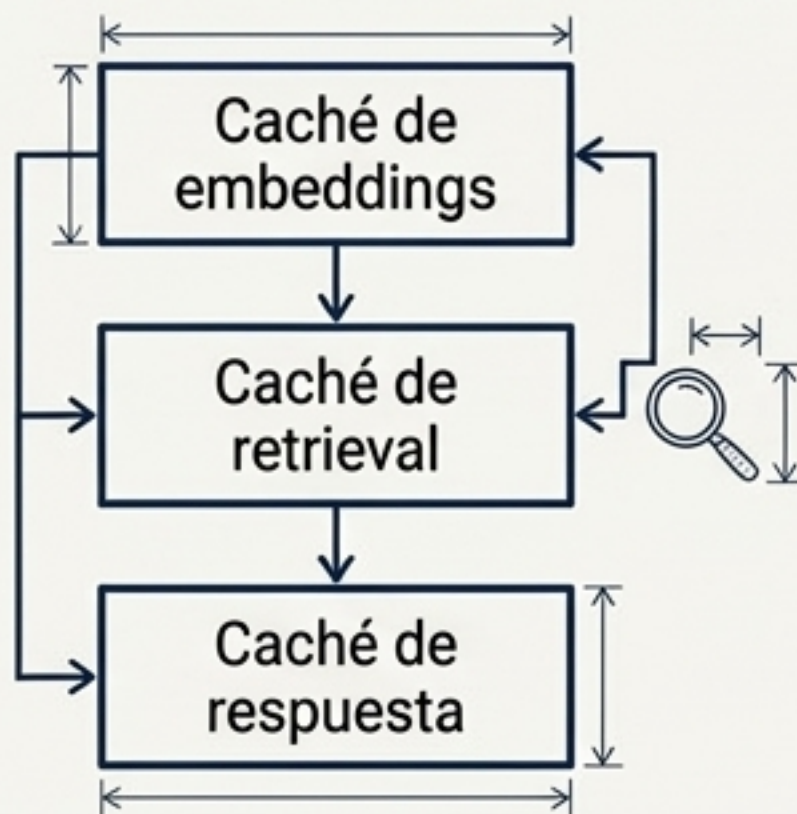


El streaming no reduce el tiempo total, pero mejora drásticamente el Time To First Token (TTFT).

Estrategias de Optimización del Pipeline



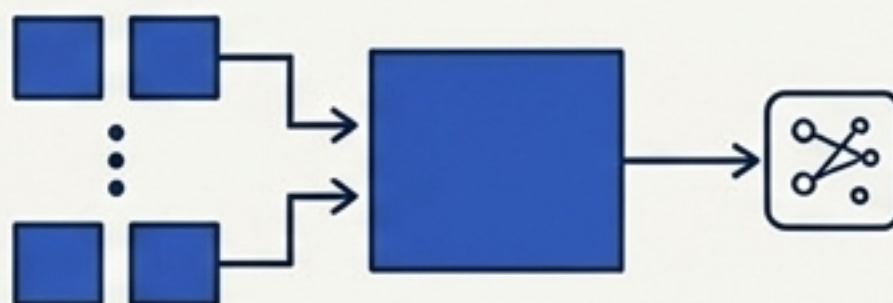
Caching e Invalidación



Peligro: Responder con datos viejos. Implementar TTL, versionado de índice o claves con versión.

Batching (Por lotes)

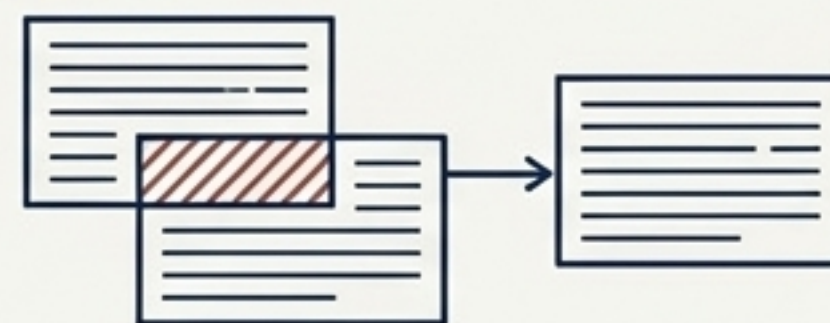
Ingestión: Agrupar embeddings de múltiples chunks.



Consulta: (Mejora throughput a costa de espera inicial).



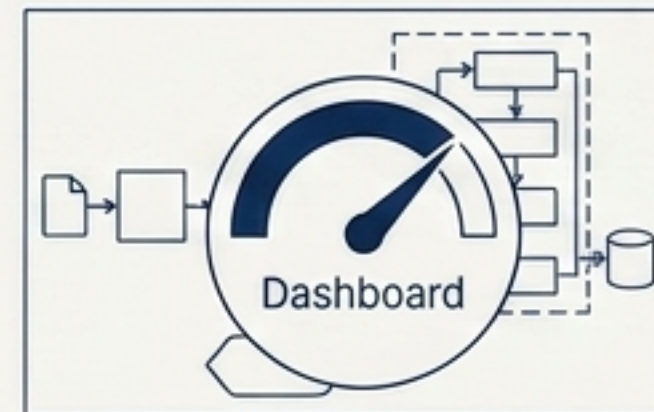
Deduplicación y Paralelismo



Eliminar chunks superpuestos reduce tokens, latencia y costo.



Evaluación: Sin un dataset, el ajuste es solo intuición



Left Cluster

Retrieval Metrics



Recall@K:
¿Apareció la evidencia relevante?



Precision@K: ¿Cuántos resultados eran realmente útiles?

MRR: ¿Qué tan arriba apareció el primer resultado correcto?



NDCG: ¿El orden completo fue apropiado?

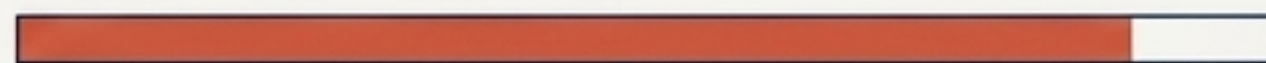
Right Cluster

Generation Metrics



Faithfulness:
¿Las afirmaciones están respaldadas?

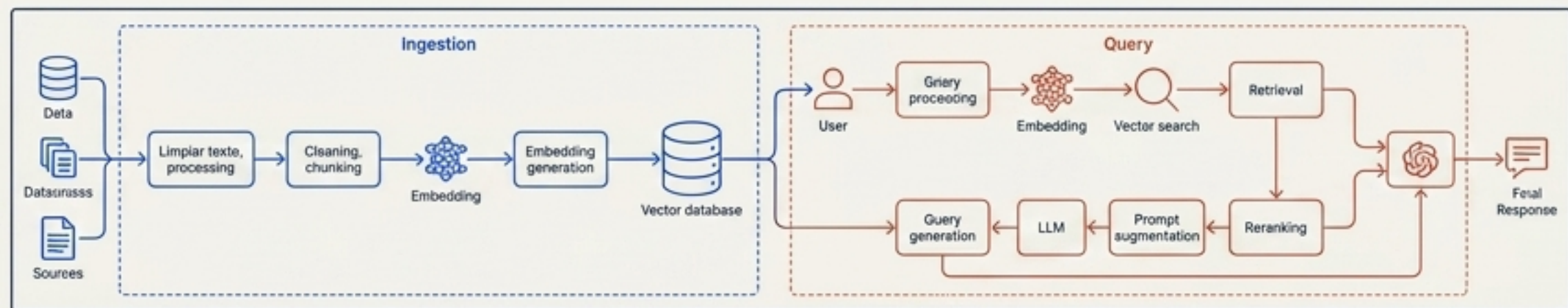
Answer relevance: ¿Contesta la pregunta?



Citation correctness: ¿Las citas coinciden con la afirmación?

El Abismo: Prototipo vs. Producción

Dimensión	Prototipo (Script Local)	Producción (Arquitectura RAG)
Datos e Índice	Archivo estático, Local (Chroma)	Fuentes dinámicas, Índice Escalable
Seguridad y Metadata	Mínima, Metadata básica	Permisos RBAC/ABAC, Metadata completa
Latencia y Errores	No controlada, Falla con excepción	Definida por SLO, Retries y Fallback
Evaluación y Trazabilidad	Manual (Print en consola)	Continua, Métricas, Trazas (Observabilidad)
Fuentes y Prompts	Fuentes opcionales, Prompt fijo	Procedencia obligatoria, Prompts versionados



Checklist Definitivo de Lanzamiento

Ingestión	Retrieval	Prompt	Producción
☐ Limpiar texto, definir chunking/overlap ☐ Generar IDs, guardar procedencia ☐ Verificar índice	☐ Mismo modelo de embedding ☐ Calibrar thresholds ☐ Aplicar filtros/permisos ☐ Deduplicar, reranear	☐ Delimitar contexto ☐ Exigir grounding/citas ☐ Política de abstención ☐ Ignorar instrucciones documentales ☐ Control de tokens	☐ Observabilidad (trazas/métricas) ☐ Estrategias de caché/invalidación ☐ Timeouts/retries, fallback ☐ Evaluación continua
Un sistema RAG confiable no es solamente un LLM conectado a una base vectorial. Es una arquitectura completa de datos.			